

Cad2024

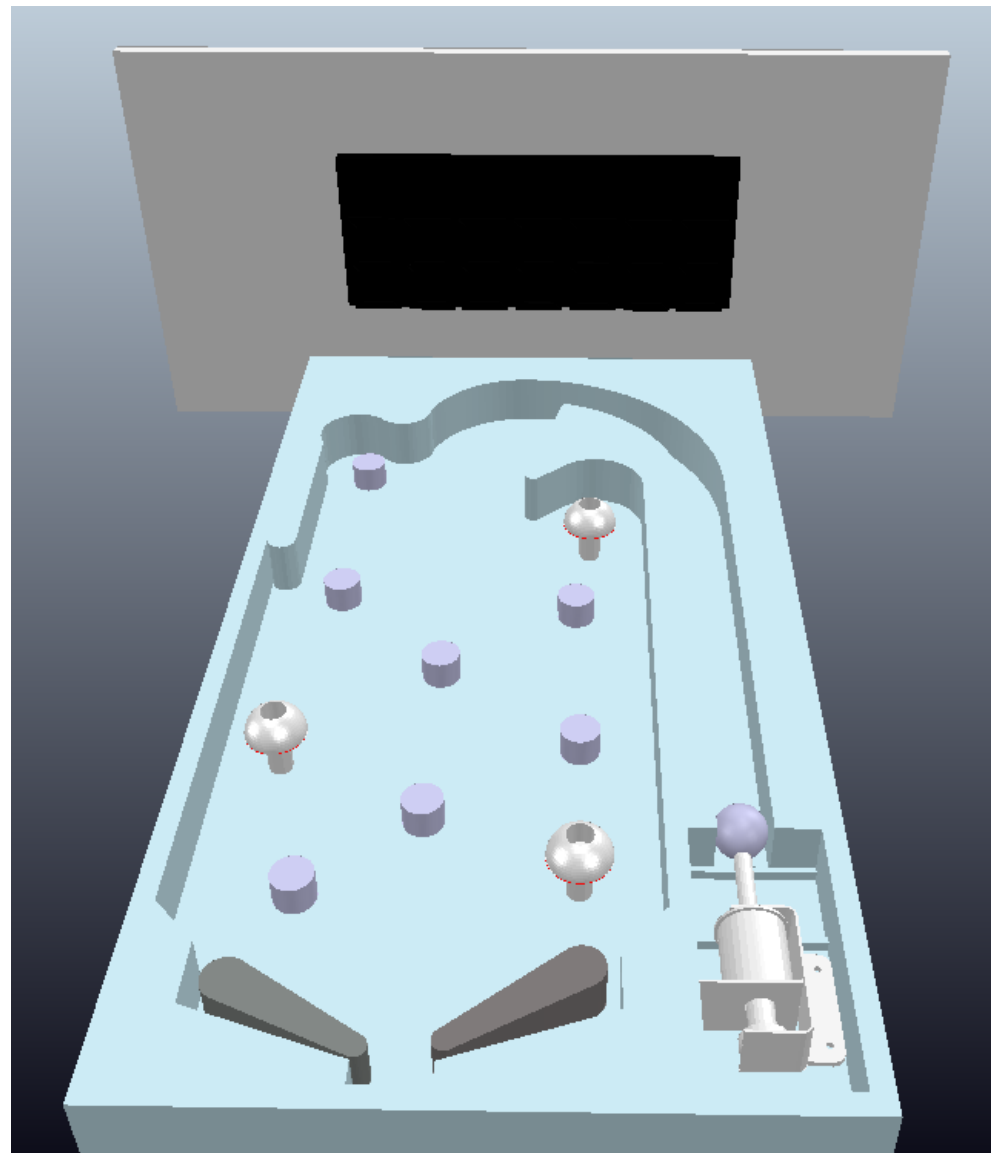
電腦輔助設計與實習

彈珠台

組長:41223122李詮聖

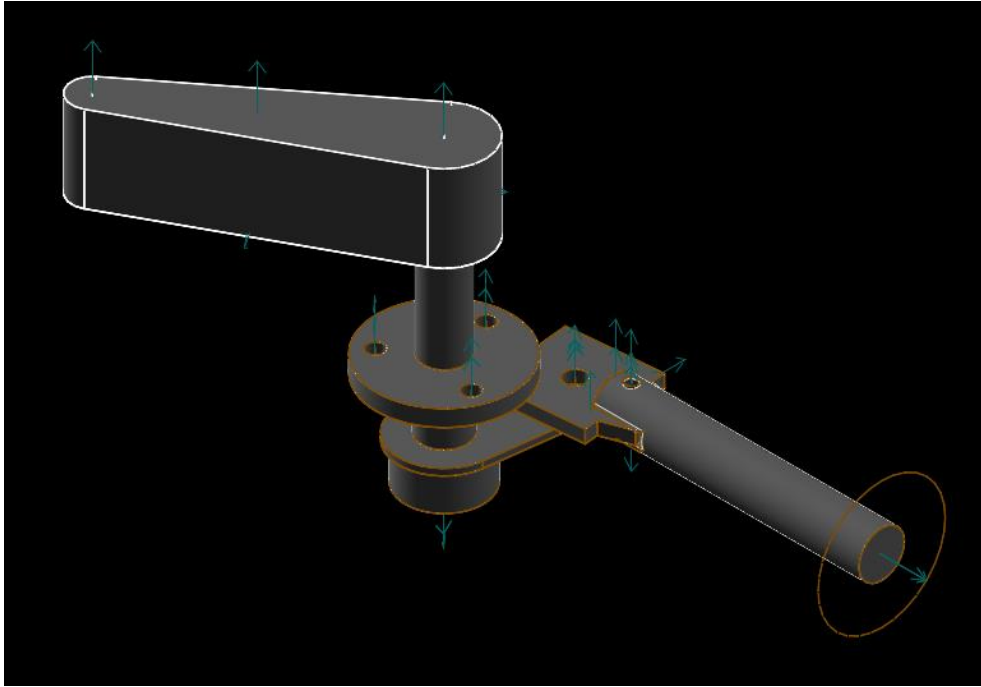
組員:41223130張翊倫

41223149謝承祐

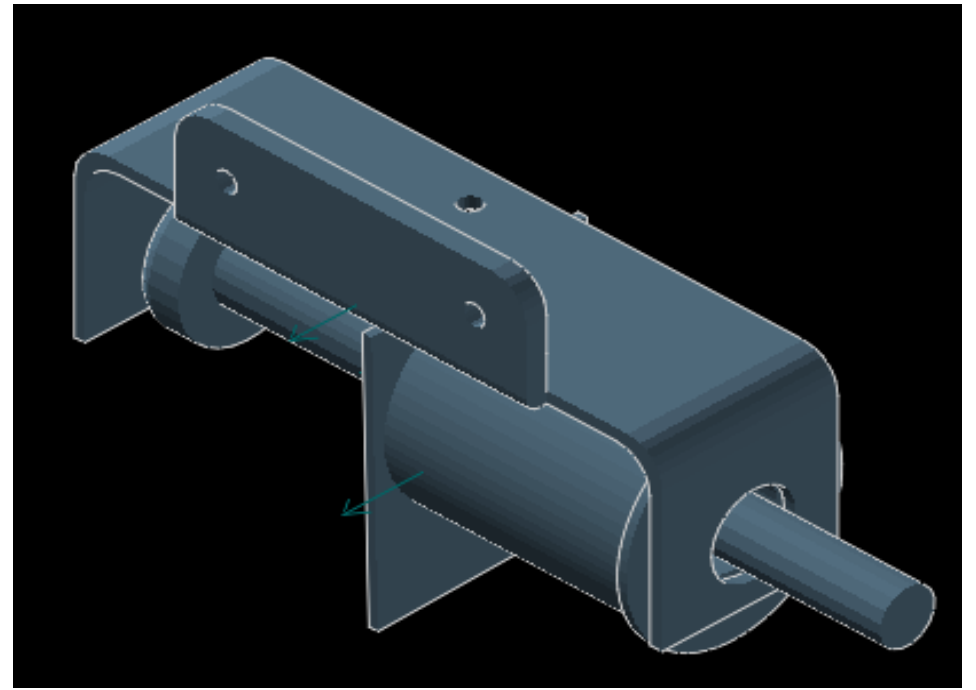


拆解

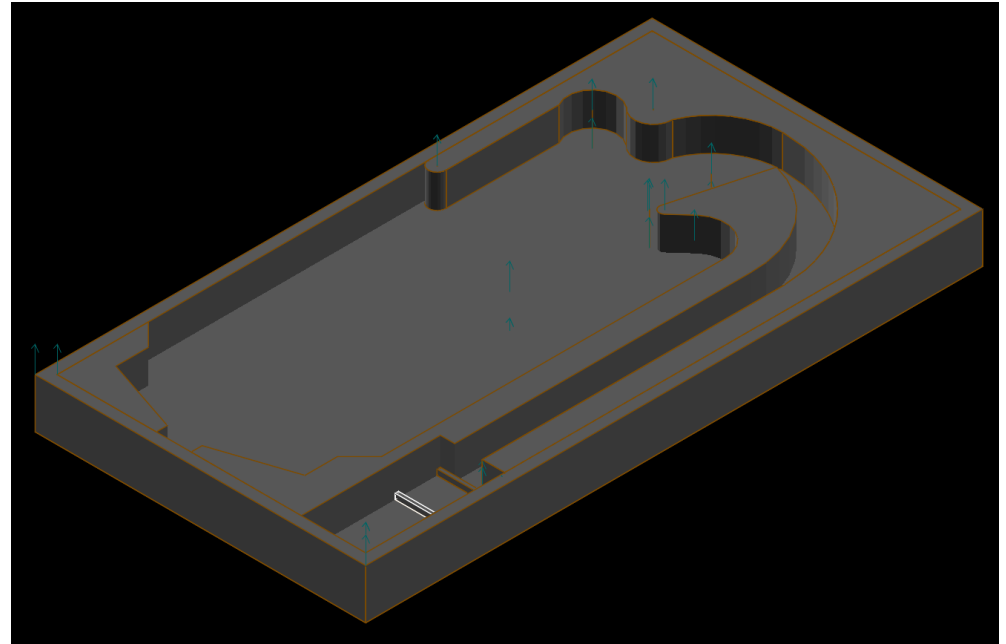
(1)撥桿



(2)發射器

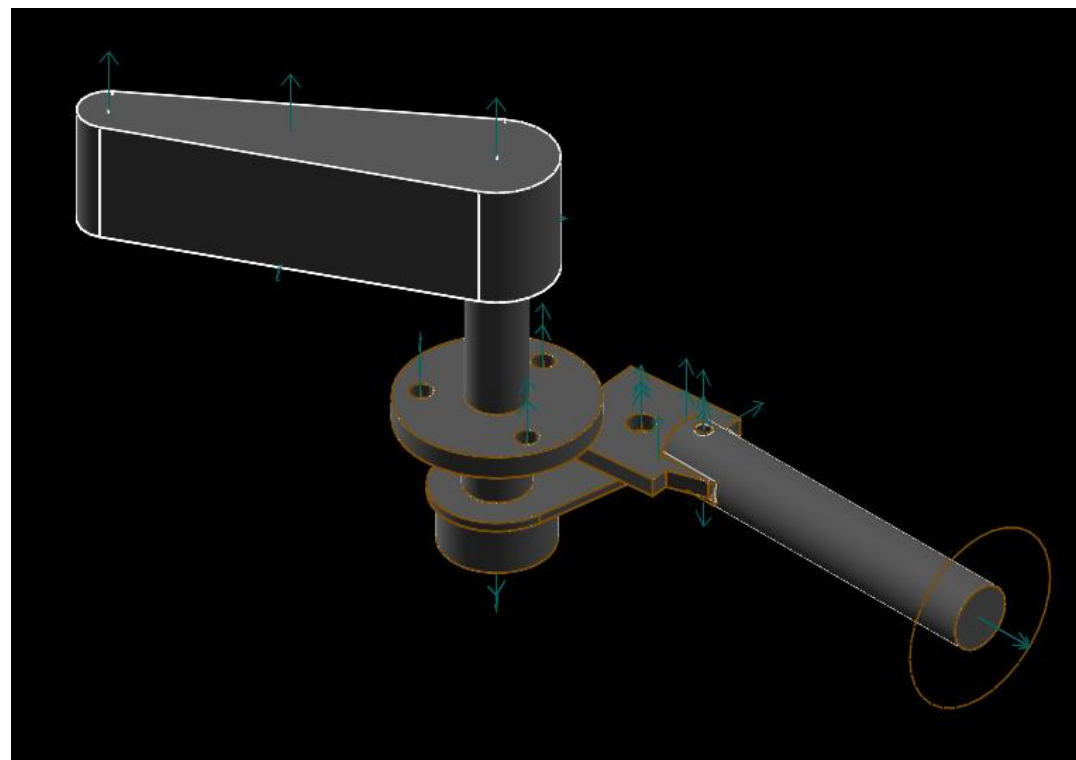


(3)彈珠檯



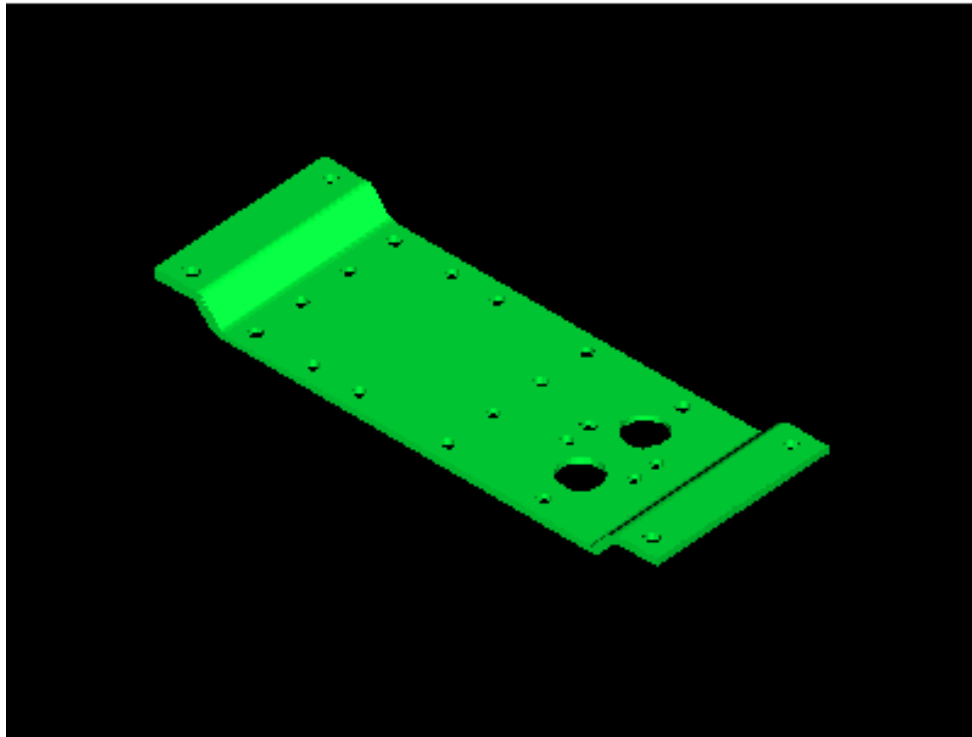
撥桿

- 1.零件分配
- 2.程式

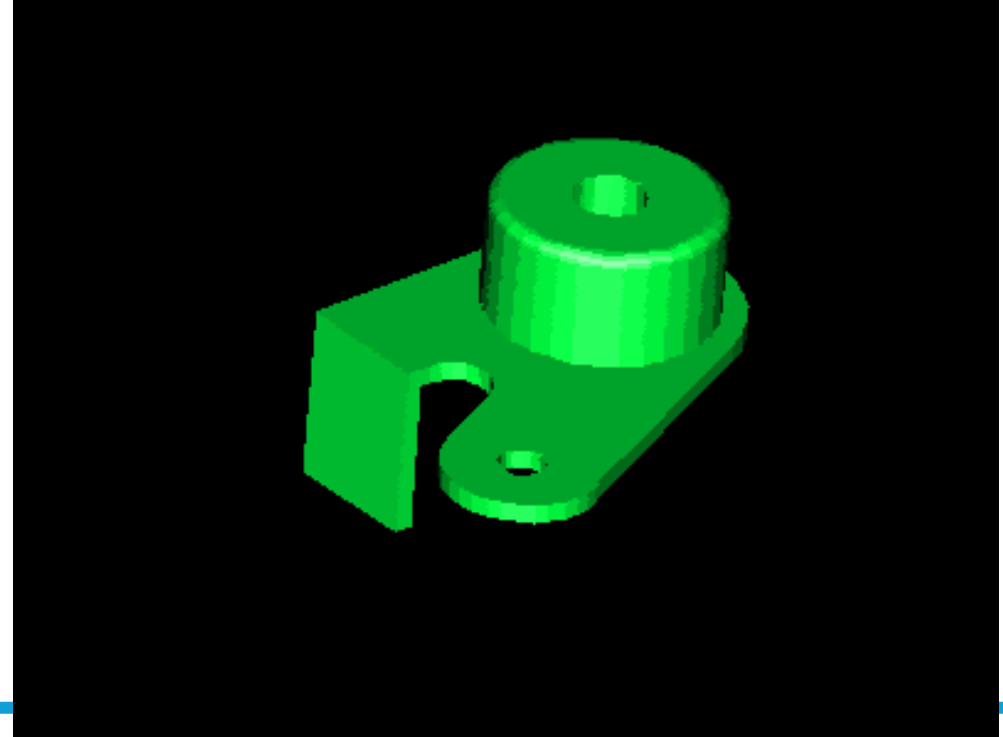


撥桿:零件分配-41223122李詮聖

Platine Batteur_sldprt

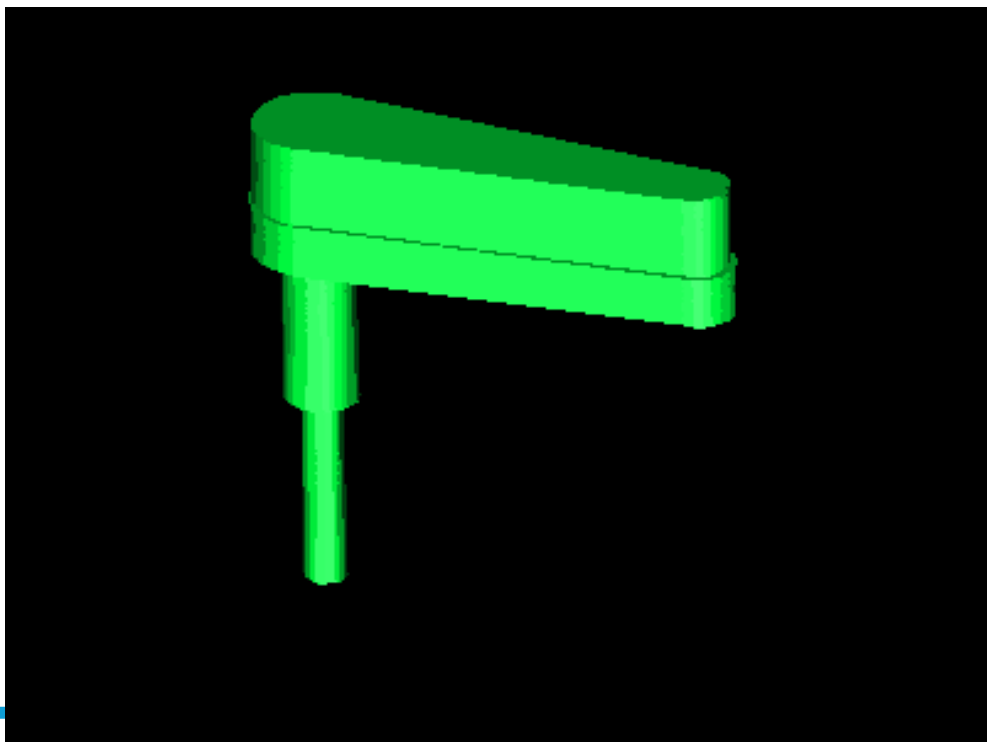


Piece métallique gauche_sldprt

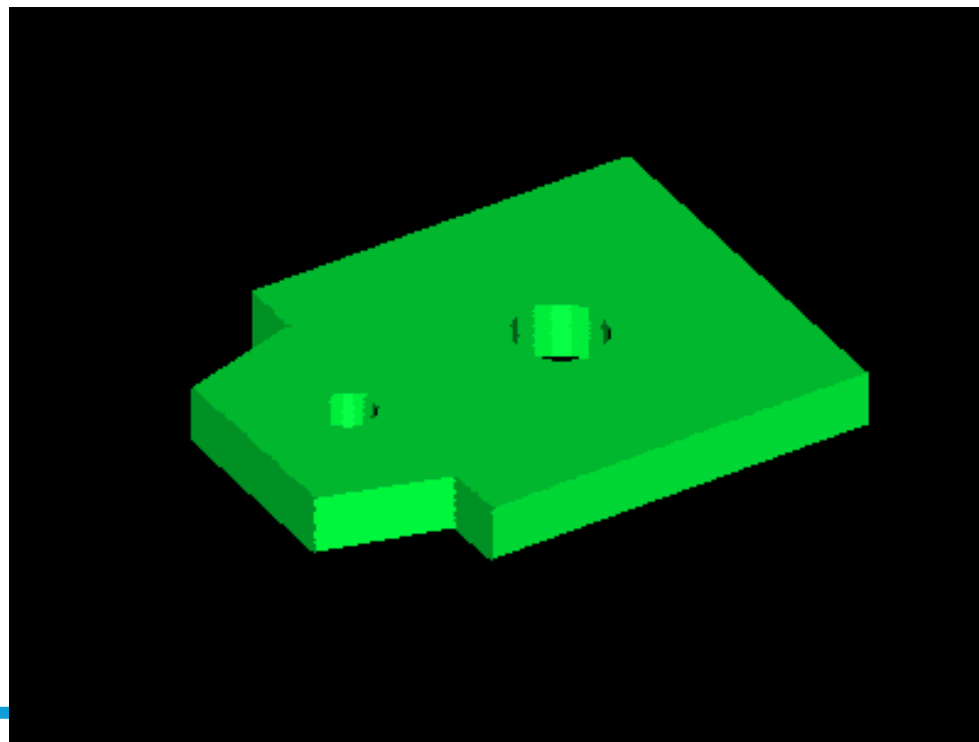


撥桿:零件分配-41223130張翊倫

Flipper_sldprt

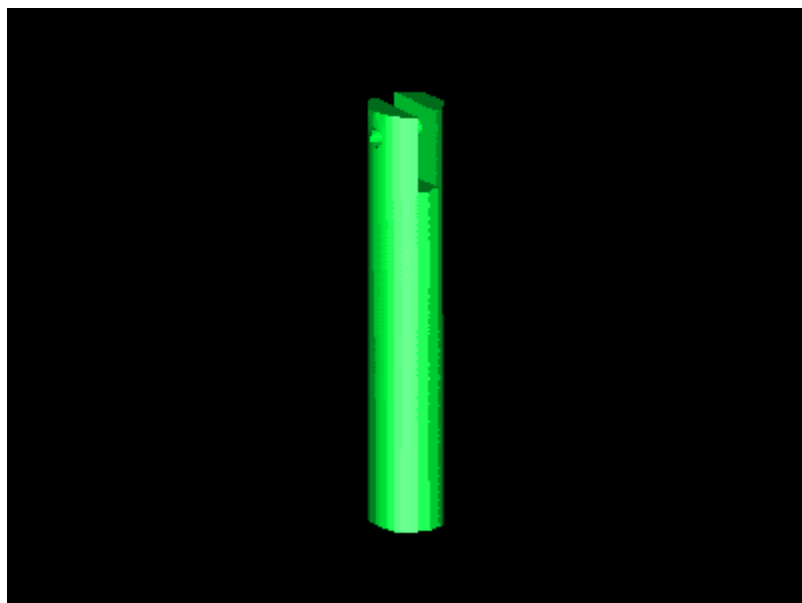


Piece composite_sldprt

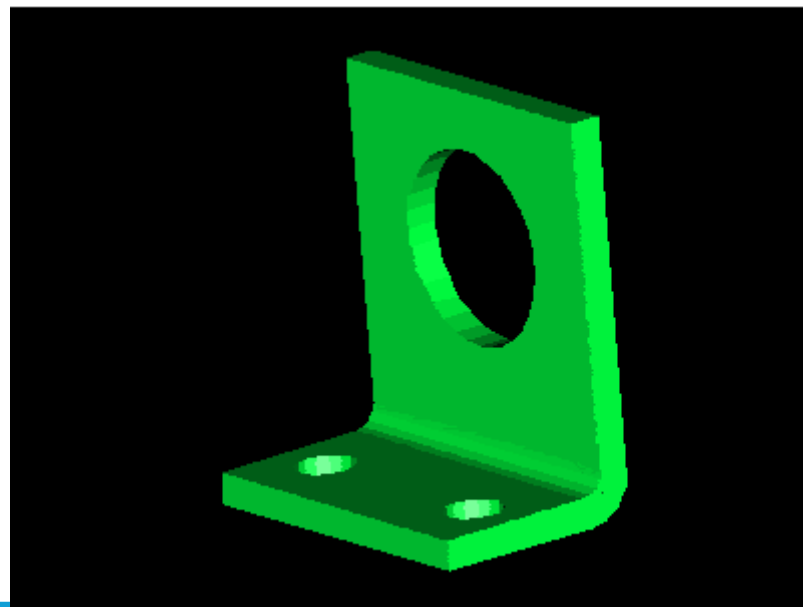


撥桿:零件分配-41223149謝承祐

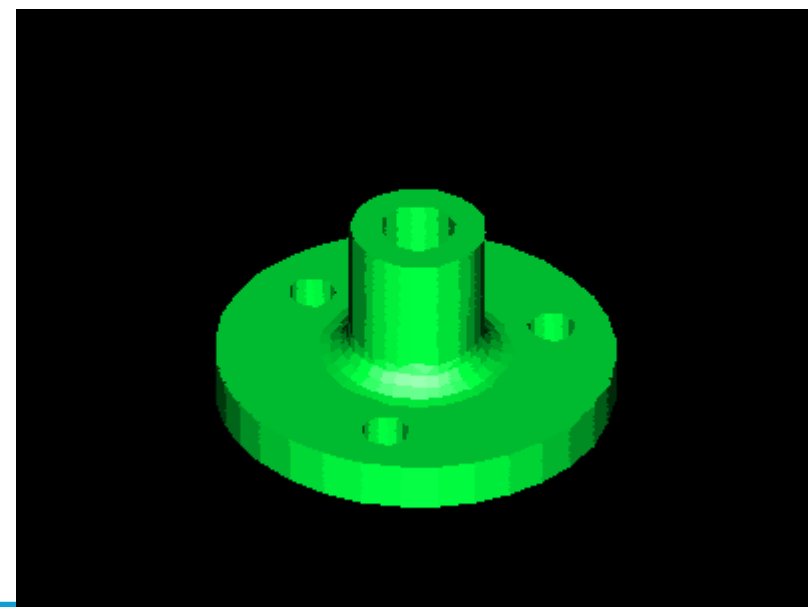
slider



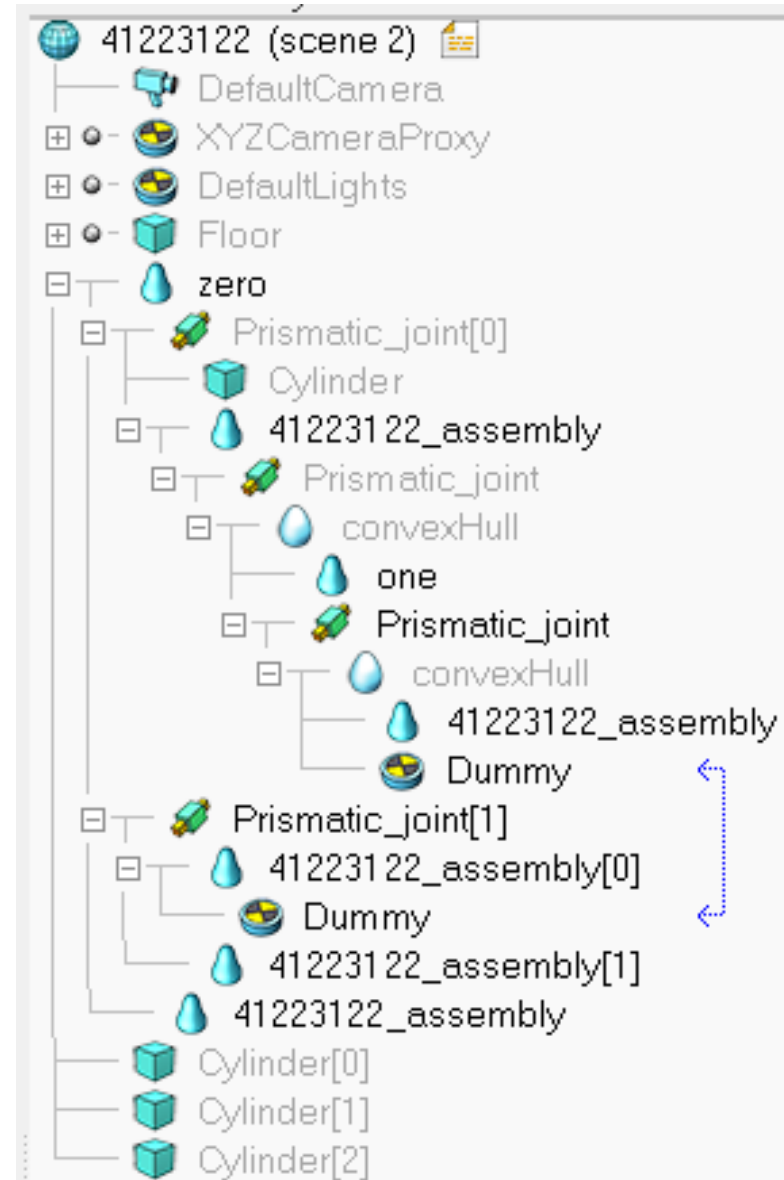
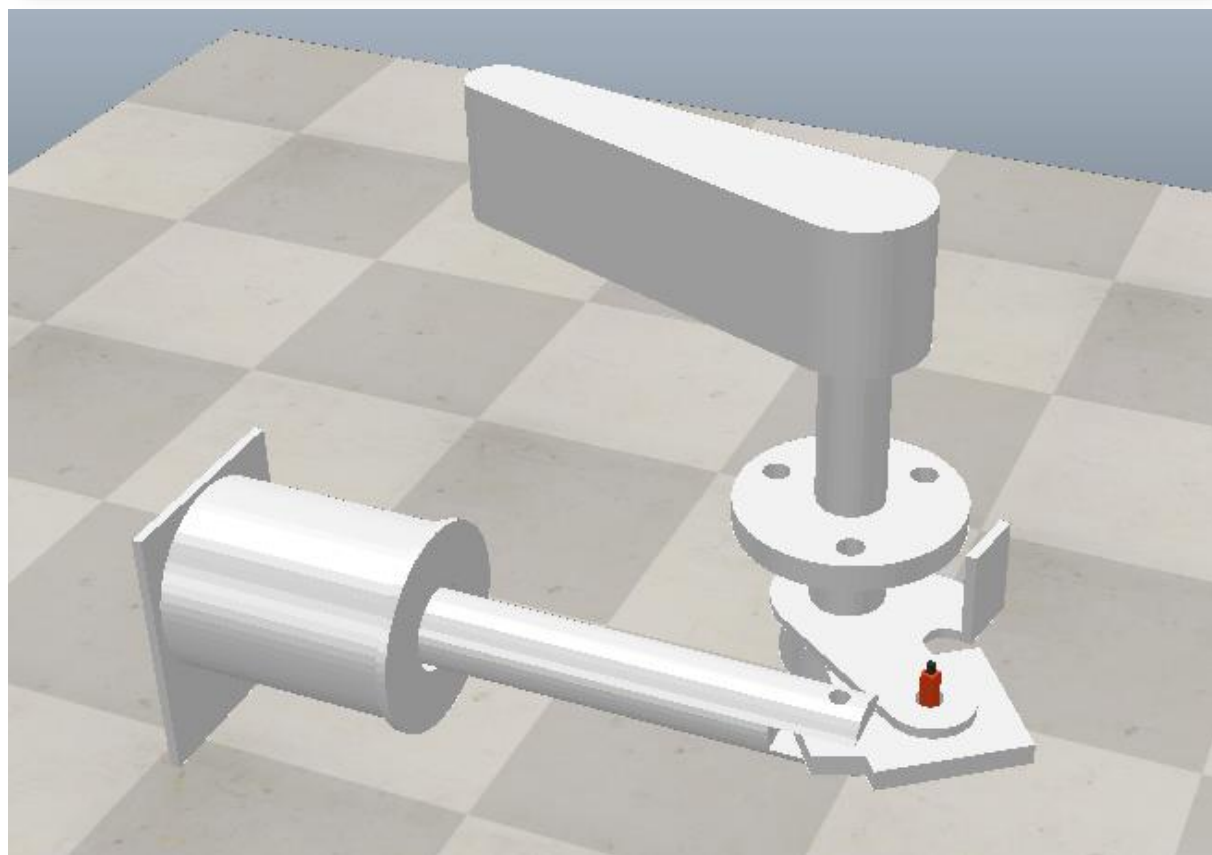
Sheet metal



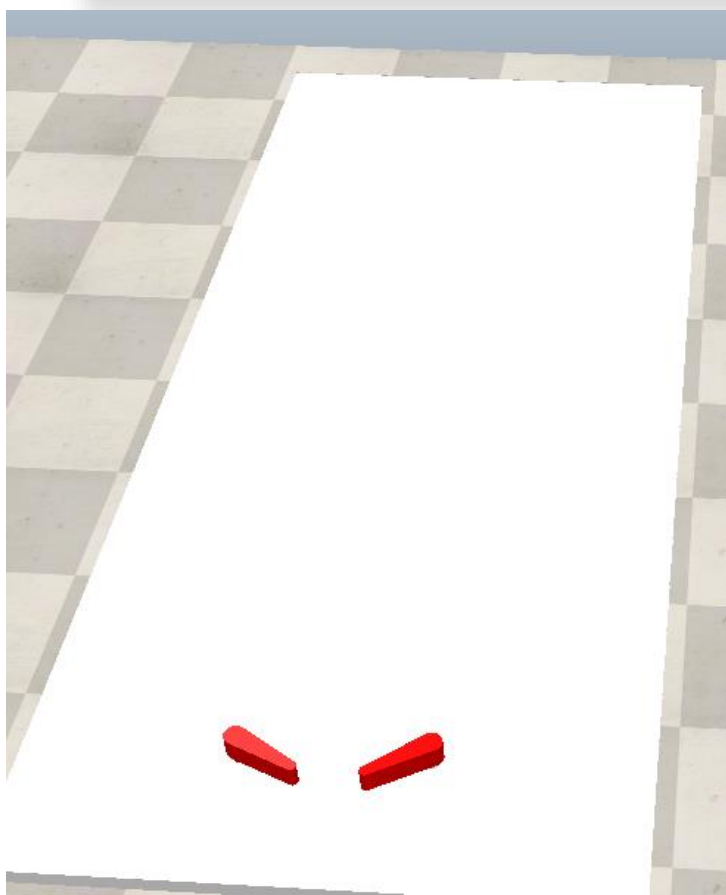
plastic_flipper



撥桿:程式



撥桿:程式



以WSPL進行

```
# pip install pyzmq cbor keyboard
from coppeliasim_zmqremoteapi_client import RemoteAPIClient
import keyboard

# Connecting to the CoppeliaSim server
client = RemoteAPIClient('localhost', 23000)

print('Program started')
sim = client.getObject('sim')

# Get the handle for the slider (prismatic joint)
cw = sim.getObject('/cw_joint')
ccw = sim.getObject('/ccw_joint')

# Starting the simulation
sim.startSimulation()
print('Simulation started')

# Main control loop
def main():
    # Keep running until simulation is stopped
    while True:
        if keyboard.is_pressed('p'): # Move slider to -0.15 position
            print("p is pressed")
            sim.setJointTargetPosition(cw, -0.25)

        if keyboard.is_pressed('l'): # Reset slider to the original position
            print("l is pressed")
            sim.setJointTargetPosition(cw, 0.0) # Reset to the initial position

        if keyboard.is_pressed('w'): # Move slider to -0.15 position
            print("w is pressed")
            sim.setJointTargetPosition(ccw, -0.28)

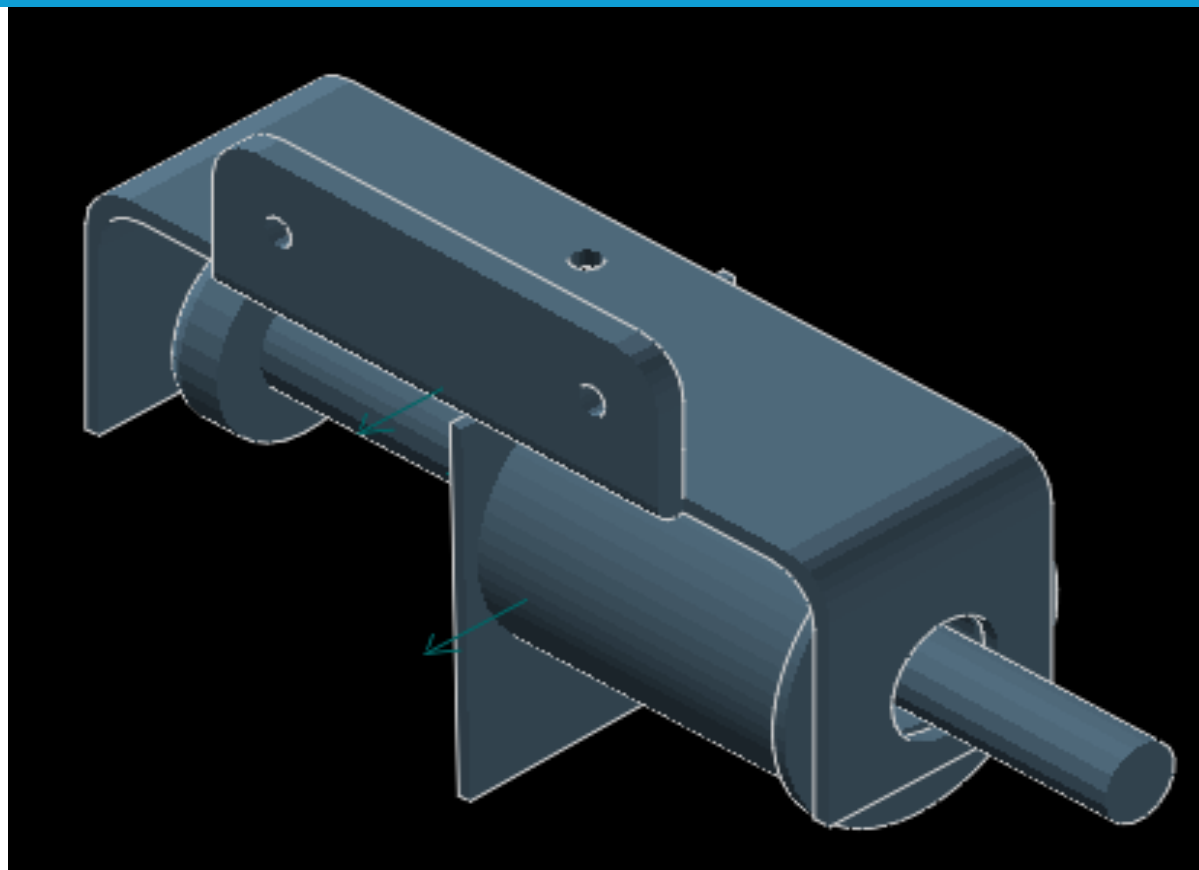
        if keyboard.is_pressed('s'): # Reset slider to the original position
            print("s is pressed")
            sim.setJointTargetPosition(ccw, 0.0) # Reset to the initial position

        if keyboard.is_pressed('t'): # Stop the simulation when 'q' is pressed
            print("t is pressed - stopping simulation")
            sim.stopSimulation()
            break

# Start the main control loop
main()
```

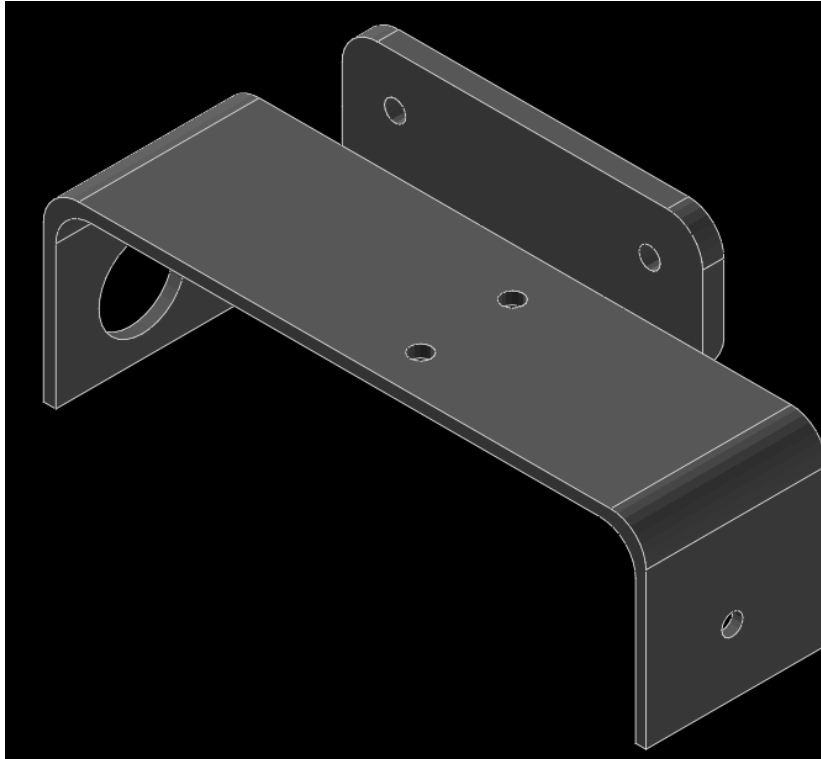
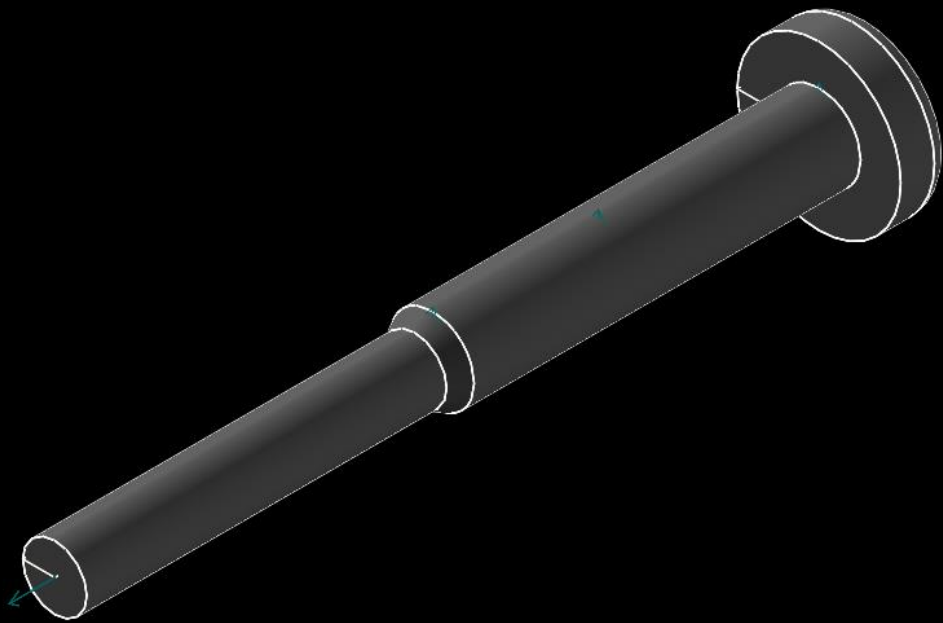
發射器

- 1.零件拆解
- 2.程式
- 3.數值調整

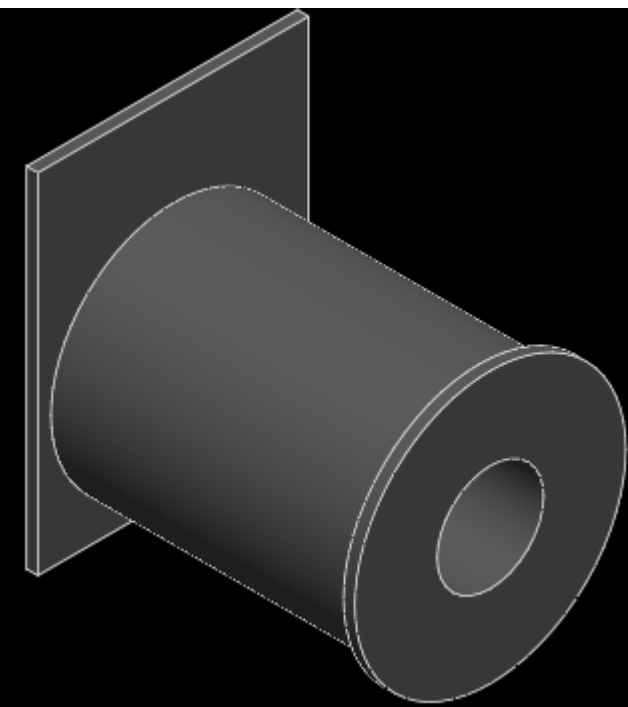


發射器:零件

Plongeur Renvoi bille_sldprt



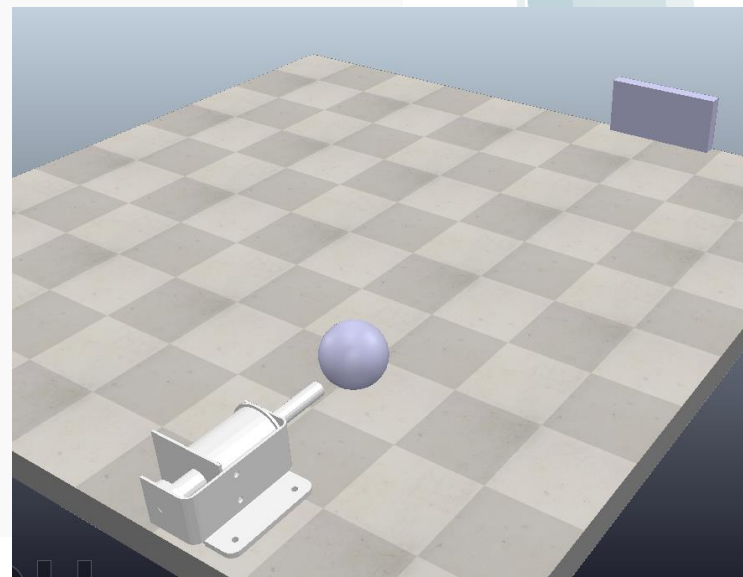
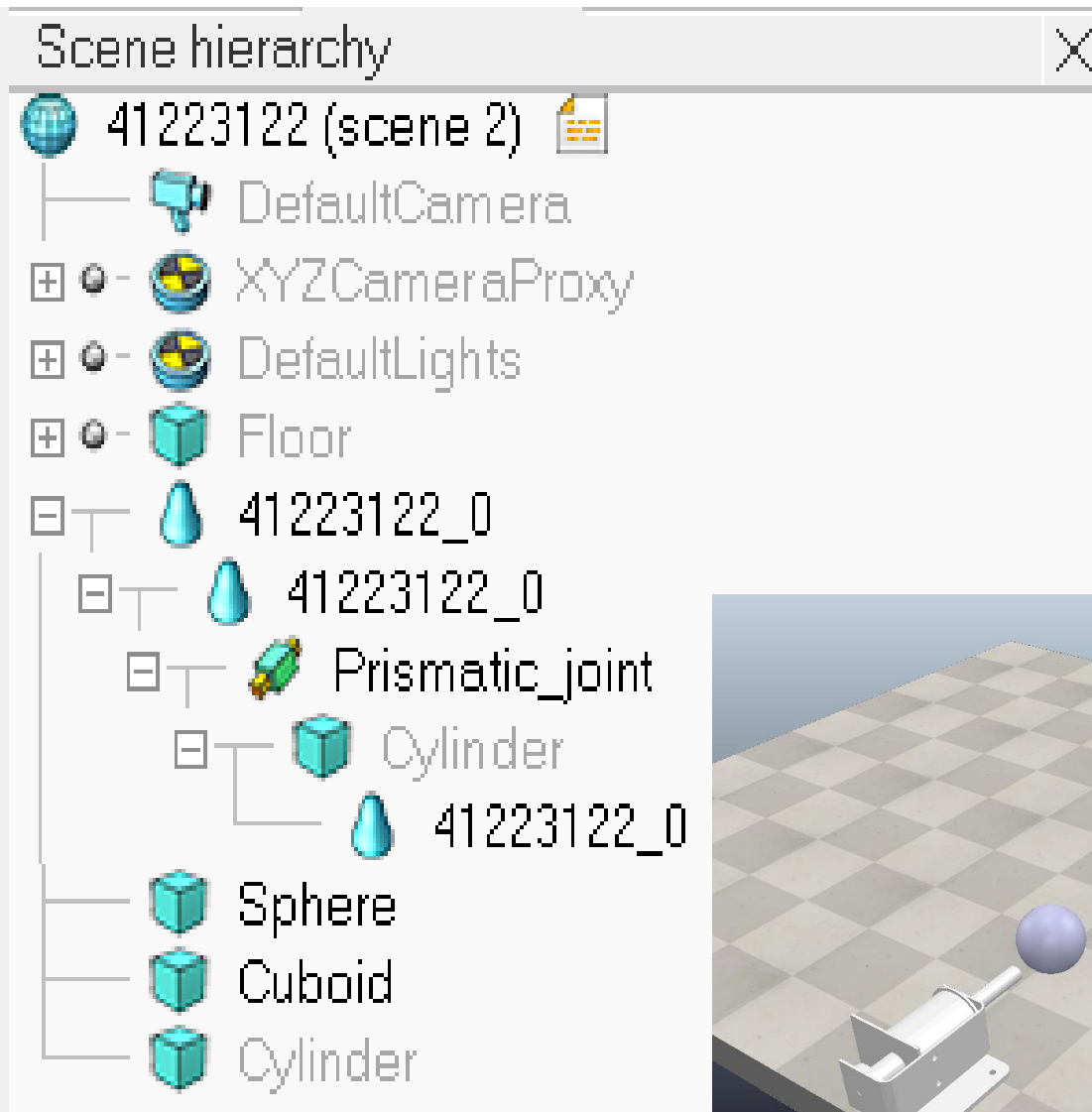
Platine renvoie bille_sldprt



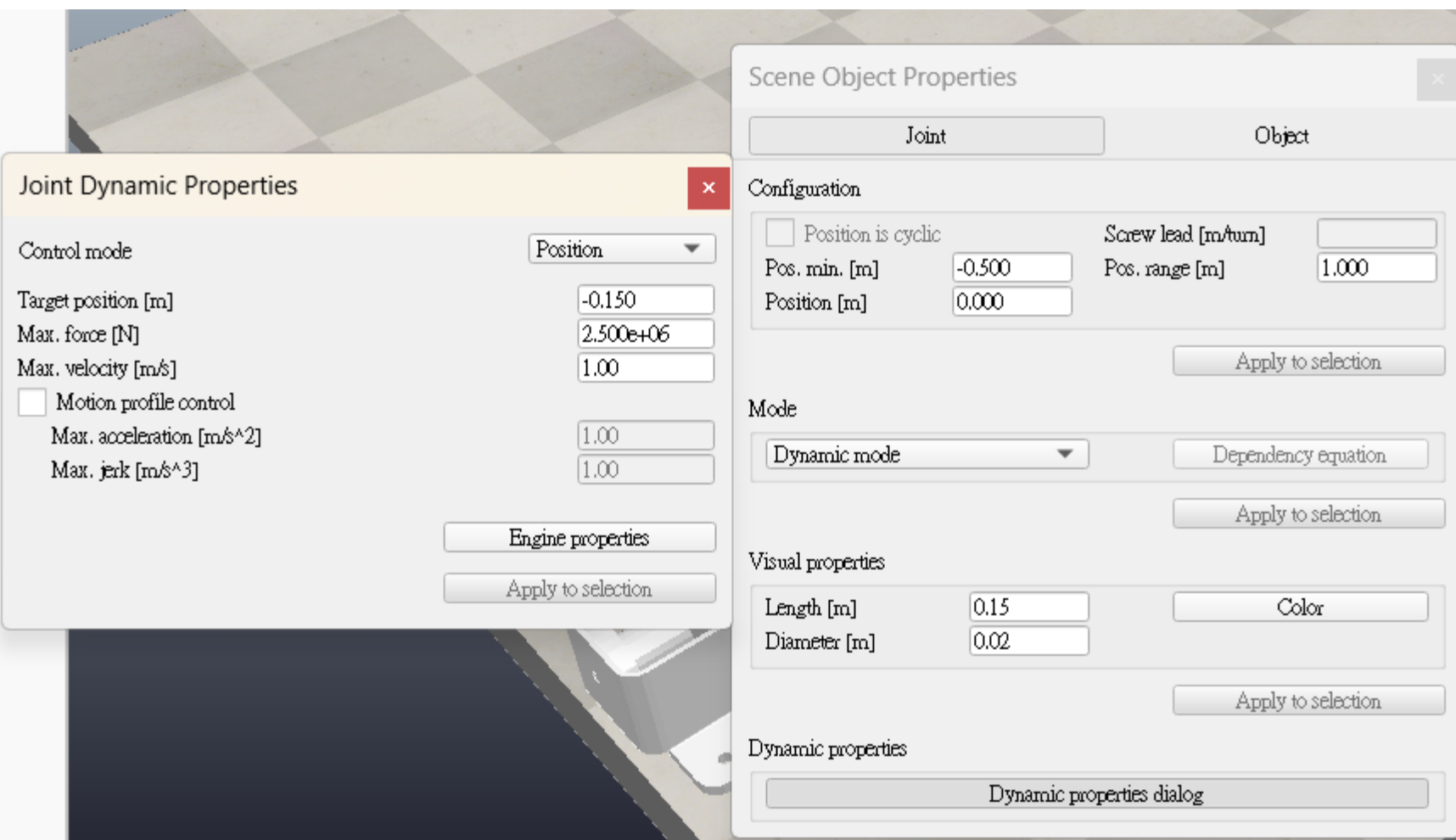
solenoid



發射器:程式



發射器:數值-桿子



1.距離1.5

*距離要取最後面到馬達距離

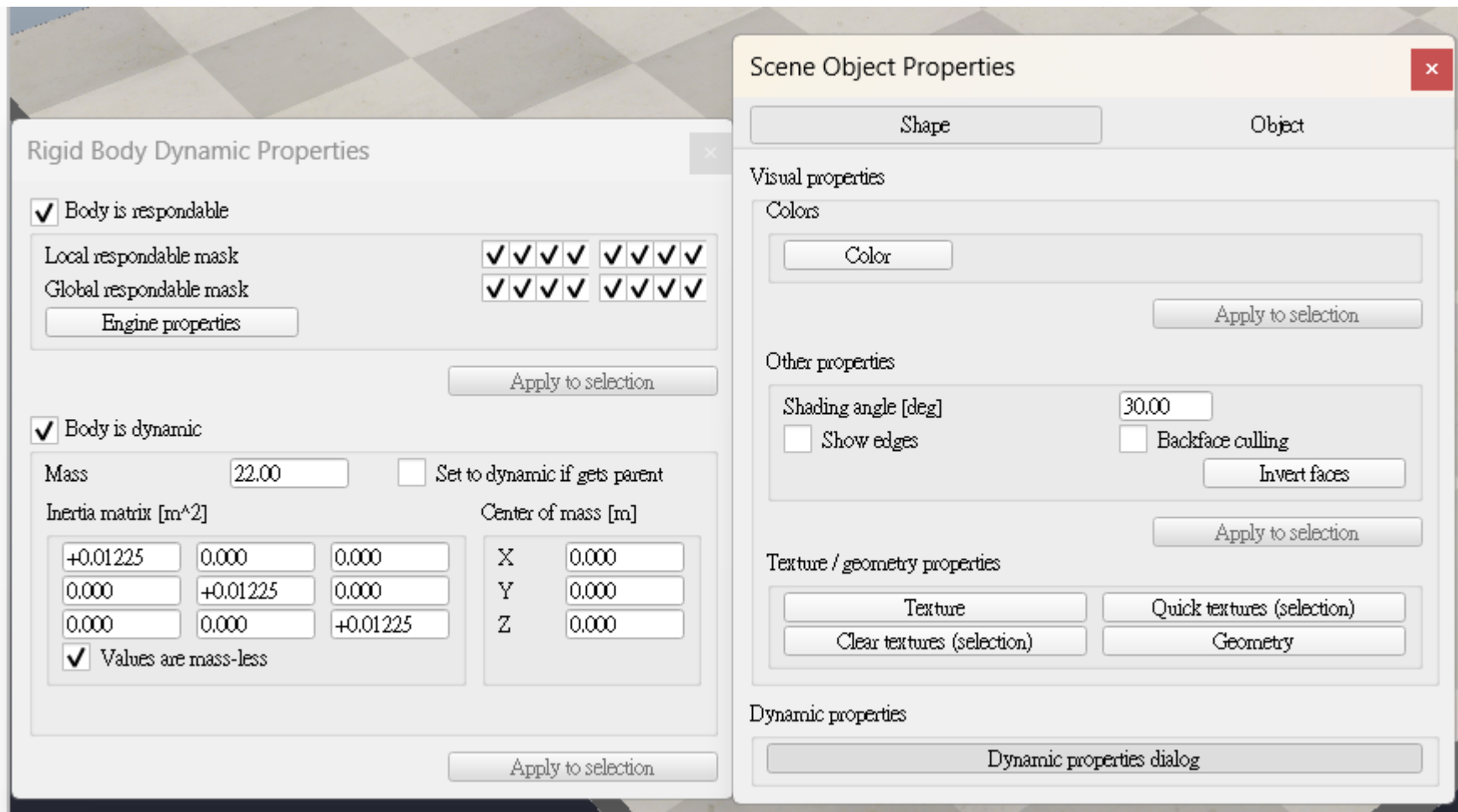
2.Force是力

*數字越大擊出的力量越大

3.velocity速度

*速度可以調整桿子擊出速度

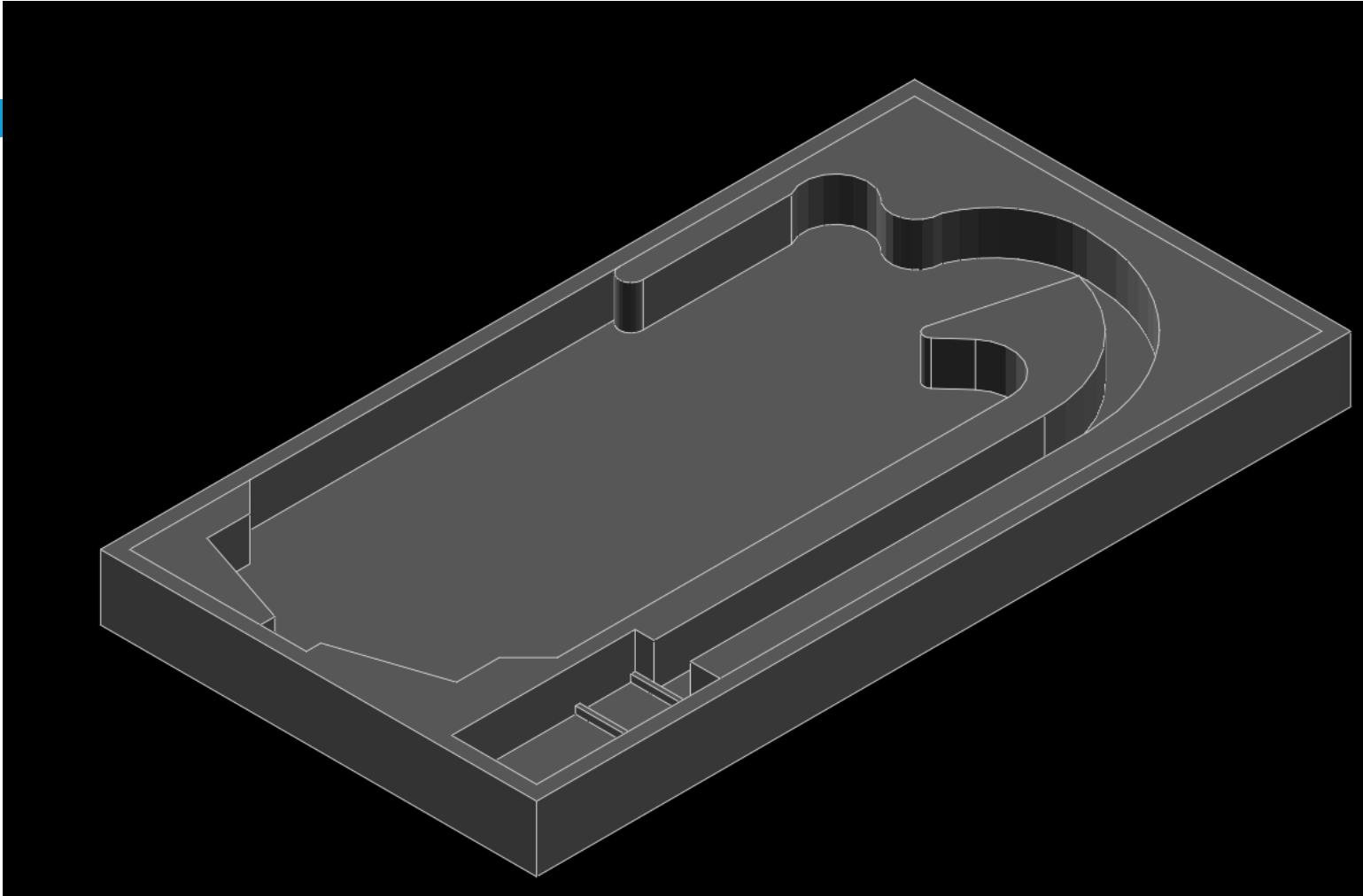
發射器:數值-Ball



*Mass是質量

*上下兩個都要勾有碰撞跟作動問題

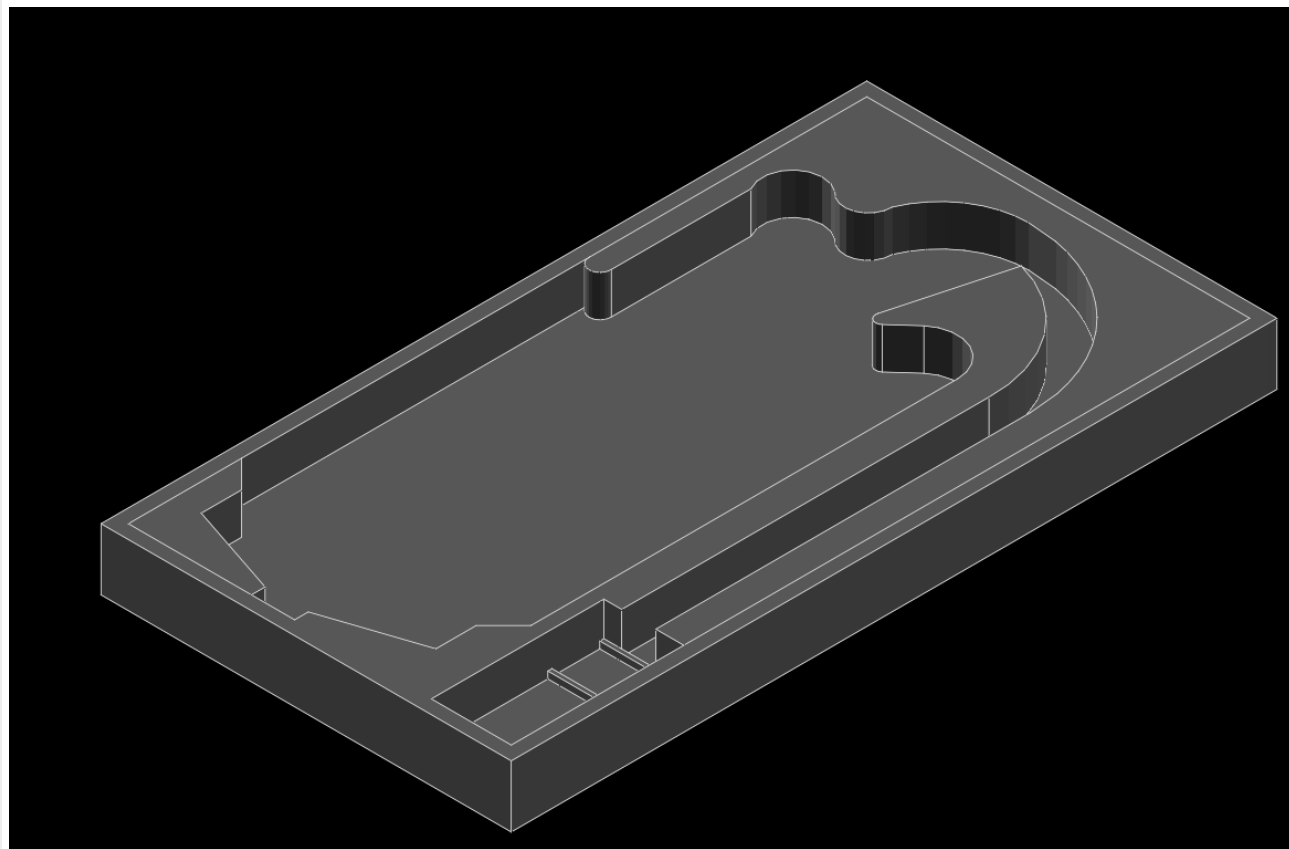
彈珠檯



- 1.設計
- 2.成品

彈珠檯:設計理念

- 1.要有通道給球進入
- 2.底面旁邊要斜坡讓球滾到撥桿



彈珠檯:成品

- 1.球會順著軌道往下
 - 2.撥桿有時候會先擊球第一下
-

未解決的問題

- 1.Bumper的回彈力太大，如果改數值太小球無法彈起，很難抓一個平均值
- 2.Bumper還是太高看起來很醜，放太低會一直彈起，未來要看看能不能拉近與檯面之間距離

